

Class Activity 2

Scenario 1: Smart City Air Quality Monitoring: A city has installed 150 air quality sensors across different locations. The sensors record PM2.5, NO₂, CO, temperature, and humidity every hour. Decision-makers struggle to identify pollution hotspots because of overlapping values Python code:

```
# Smart City Air Quality Monitoring Visualization
```

```
import pandas as pd import
```

```
numpy as np import
```

```
matplotlib.pyplot as plt
```

```
# Generate sample data np.random.seed(10)
```

```
n = 150
```

```
data = pd.DataFrame({  
    "Sensor_ID": range(1, n+1),  
    "X": np.random.randint(0, 100, n),  
    "Y": np.random.randint(0, 100, n),  
    "PM25": np.random.randint(20, 250, n),  
    "NO2": np.random.randint(10, 150, n),  
    "Temperature": np.random.randint(18, 42, n),  
    "Humidity": np.random.randint(30, 95, n)  
})
```

```
print(data.head())
```

```
#####
```

```
# 1. Scatter Plot (Cartesian Coordinates)
```

```
#####
```

```
plt.figure(figsize=(8,6)) plt.scatter(data["X"],  
data["Y"], color='blue') plt.title("Sensor  
Locations") plt.xlabel("X Coordinate")  
plt.ylabel("Y Coordinate") plt.grid(True)  
plt.show()
```

```
#####
```

```
# 2. PM2.5 Quantitative Color Palette
```

```
#####
```

```
plt.figure(figsize=(8,6))  
plt.scatter(data["X"], data["Y"],  
c=data["PM25"],  
cmap="viridis", s=120)
```

```
plt.colorbar(label="PM2.5")  
plt.title("PM2.5 Pollution Levels")  
plt.xlabel("X") plt.ylabel("Y")  
plt.show()
```

```
#####
```

```
#####
```

```
# 3. NO2 Diverging Color Palette
```

```
#####
```

```
plt.figure(figsize=(8,6))
plt.scatter(data["X"], data["Y"],
c=data["NO2"],      cmap="coolwarm",
s=120)
```

```
plt.colorbar(label="NO2")
plt.title("NO2 Levels")
plt.xlabel("X")
plt.ylabel("Y") plt.show()
```

```
#####
```

4. Temperature Palette

```
#####
```

```
plt.figure(figsize=(8,6))
plt.scatter(data["X"], data["Y"],
c=data["Temperature"],
cmap="plasma",      s=120)
```

```
plt.colorbar(label="Temperature") plt.title("Temperature
Distribution") plt.xlabel("X") plt.ylabel("Y") plt.show()
```

```
#####
```

5. Humidity Palette

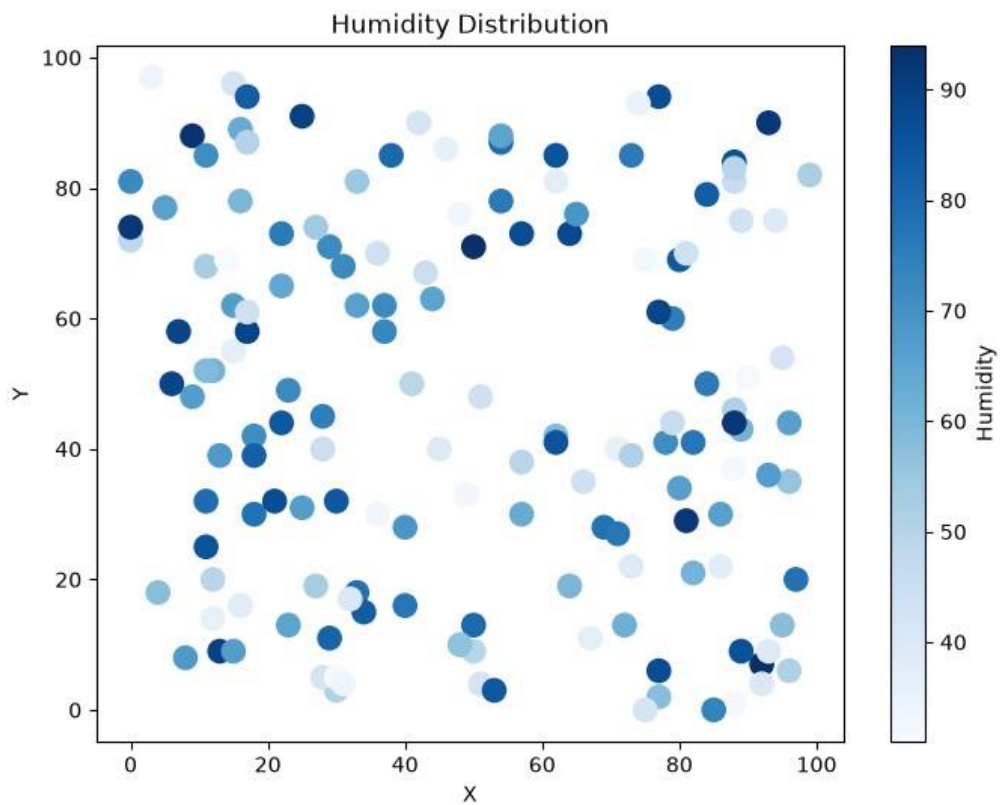
```
#####
```

```
plt.figure(figsize=(8,6))
plt.scatter(data["X"], data["Y"],
```

```
c=data["Humidity"],  
cmap="Blues",      s=120)
```

```
plt.colorbar(label="Humidity")  
plt.title("Humidity Distribution")  
plt.xlabel("X") plt.ylabel("Y")  
plt.show()
```

output:



R code:

```
# Smart City Air Quality Monitoring
```

```
library(ggplot2)
```

```
set.seed(10)
```

```
n <- 150
```

```
data <- data.frame(  
  Sensor_ID = 1:n,  
  X = sample(0:100, n, replace=TRUE),  
  Y = sample(0:100, n, replace=TRUE),  
  PM25 = sample(20:250, n, replace=TRUE),  
  NO2 = sample(10:150, n, replace=TRUE),  
  Temperature = sample(18:42, n, replace=TRUE),  
  Humidity = sample(30:95, n, replace=TRUE)  
)
```

```
head(data)
```

```
#####
```

```
# 1. Sensor Locations
```

```
#####
```

```
ggplot(data, aes(X, Y)) + geom_point(color="blue",  
size=3) + ggtitle("Sensor Locations")
```

```
#####
```

```
# 2. PM2.5 Quantitative Palette
```

```
#####
```

```
ggplot(data, aes(X, Y, color=PM25)) +  
geom_point(size=4) +
```

```
scale_color_viridis_c() + ggtitle("PM2.5  
Levels")
```

```
#####
```

```
# 3. NO2 Diverging Palette
```

```
##### ggplot(data, aes(X, Y, color=NO2))  
+ geom_point(size=4) + scale_color_gradient2(low="blue",  
mid="white",  
high="red",  
midpoint=80) + ggtitle("NO2  
Levels")
```

```
#####
```

```
# 4. Temperature Distribution
```

```
#####
```

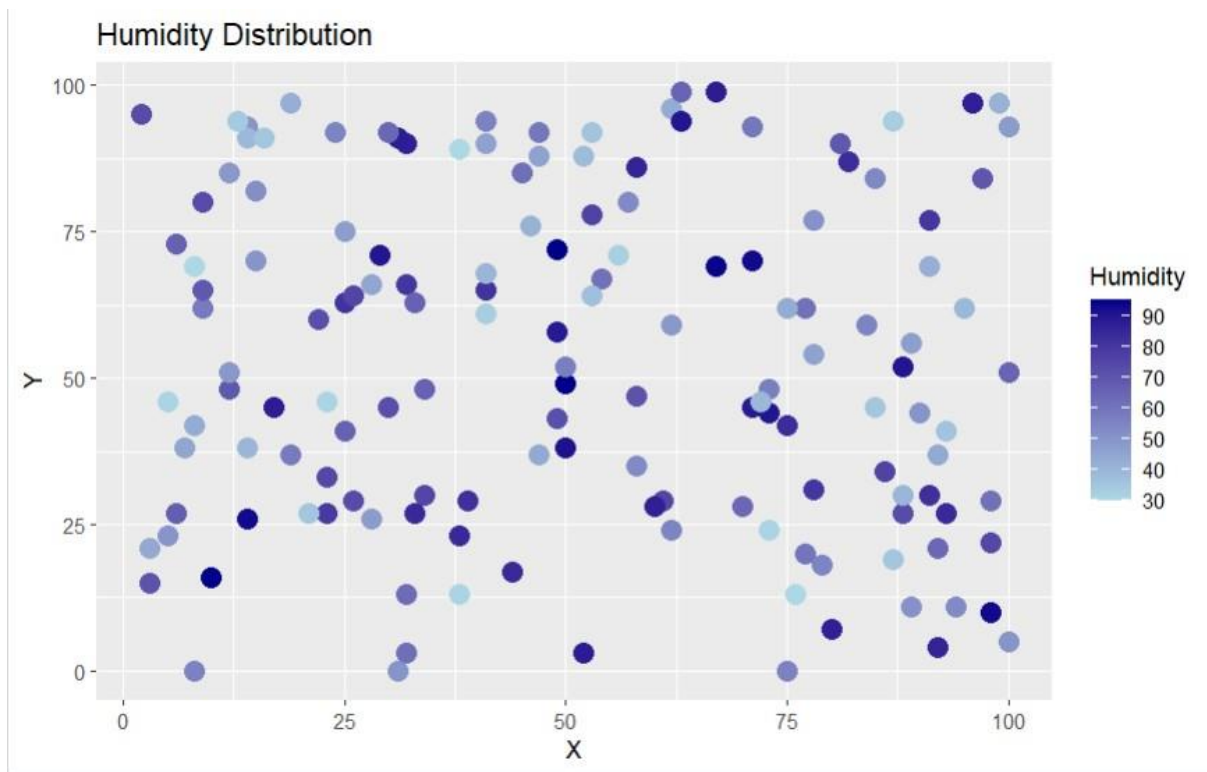
```
ggplot(data, aes(X, Y, color=Temperature)) +  
geom_point(size=4) + scale_color_gradient(low="yellow",  
high="red") + ggtitle("Temperature Distribution")
```

```
#####
```

```
# 5. Humidity Distribution
```

```
#####
```

```
ggplot(data, aes(X, Y, color=Humidity)) +  
geom_point(size=4) +  
scale_color_gradient(low="lightblue", high="darkblue") +  
ggtitle("Humidity Distribution") output:
```



Scenario 2: Earthquake Risk Analysis: A geological survey contains earthquake magnitude (0.5–9.5), depth (1–700 km), latitude, longitude, and aftershock frequency. Small earthquakes dominate the dataset, making large events difficult to interpret.

Python code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Generate sample earthquake dataset
np.random.seed(10)

n = 300

data = pd.DataFrame({
    "Magnitude": np.random.uniform(0.5, 9.5, n),
    "Depth": np.random.uniform(1, 700, n),
    "Latitude": np.random.uniform(-90, 90, n),
    "Longitude": np.random.uniform(-180, 180, n),
    "Aftershocks": np.random.randint(1, 1000, n)
})

print(data.head())
```

```
#####
#           1.           Linear           Scale           Scatter           Plot
#####
```

```
plt.figure(figsize=(8,6))
plt.scatter(data["Magnitude"],
data["Aftershocks"],      color="blue")
```

```
plt.title("Earthquake Magnitude vs Aftershocks (Linear Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency") plt.grid(True)
plt.show()
```

```
#####
#           2.           Logarithmic           Scale           Scatter           Plot
#####
```

```
plt.figure(figsize=(8,6)) plt.scatter(data["Magnitude"],
data["Aftershocks"],
      color="red")
```

```
plt.yscale("log")
```

```
plt.title("Earthquake Magnitude vs Aftershocks (Log Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency (Log Scale)") plt.grid(True) plt.show()
```

```
#####
#           3.           Magnitude           vs           Depth
#####
```

```
plt.figure(figsize=(8,6)) plt.scatter(data["Magnitude"],
      data["Depth"],
c=data["Magnitude"],      cmap="viridis",
      s=80)
```

```
plt.colorbar(label="Magnitude")
```

```
plt.title("Magnitude vs plt.xlabel("Magnitude")           Depth")
```



```
plt.ylabel("Depth (km)")
plt.grid(True) plt.show()
```

```
#####
```

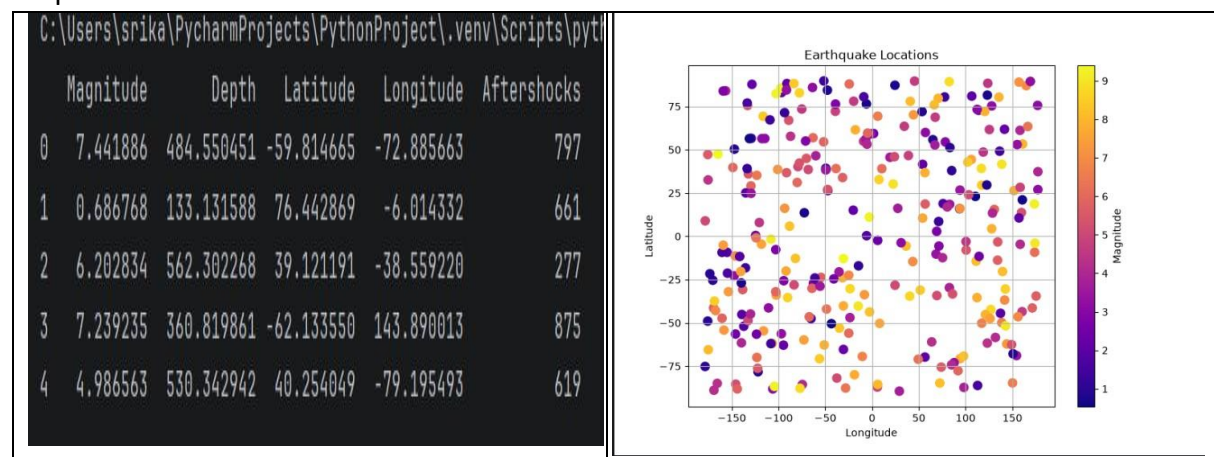
```
# 4. Earthquake Locations
#####
```

```
plt.figure(figsize=(8,6)) plt.scatter(data["Longitude"],
    data["Latitude"],
    c=data["Magnitude"],      cmap="plasma",
    s=70)
```

```
plt.colorbar(label="Magnitude")
```

```
plt.title("Earthquake Locations")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.grid(True) plt.show()
```

output:



R code:

```
# Earthquake Risk Analysis
```

```
set.seed(10)
```

```
n <- 300
```

```
Magnitude <- runif(n,0.5,9.5)
```

```
Depth <- runif(n,1,700)
```

```
Latitude <- runif(n,-90,90)
```

```
Longitude <- runif(n,-180,180)
```

```
Aftershocks <- sample(1:1000,n,replace=TRUE)
```

```
#####
```

```
# 1. Linear Scale
```

```
#####
```

```
plot(Magnitude, Aftershocks, pch=19,  
col="blue", main="Magnitude vs Aftershocks  
(Linear Scale)", xlab="Magnitude",  
ylab="Aftershock Frequency")
```

```
#####
```

```
# 2. Logarithmic Scale
```

```
##### plot(Magnitude, Aftershocks,  
log="y", pch=19, col="red", main="Magnitude vs Aftershocks (Log Scale)",  
xlab="Magnitude", ylab="Aftershock Frequency (Log Scale)")
```

```
#####
```

```
# 3. Magnitude vs Depth
```

```
#####
```

```
colors <- heat.colors(n)
```

```
plot(Magnitude,
      Depth,    pch=19,
      col=colors[rank(Magnitude)],
      main="Magnitude vs Depth",
      xlab="Magnitude",
      ylab="Depth (km)")
```

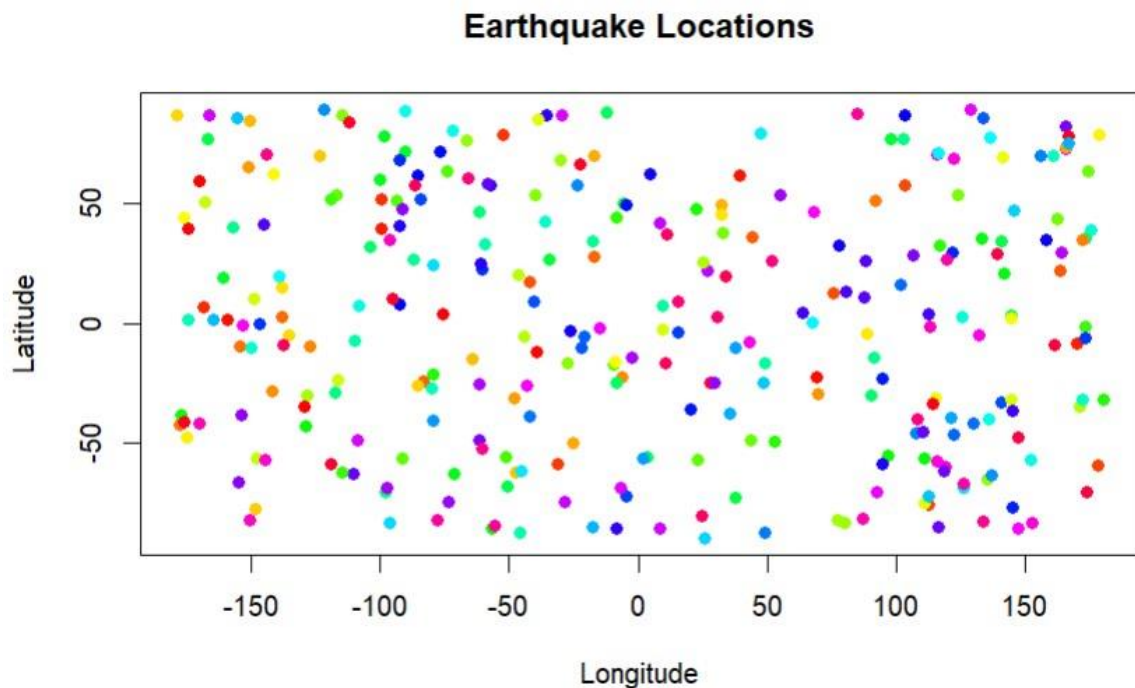
```
#####
```

```
# 4. Earthquake Locations
```

```
#####
```

```
colors <- rainbow(n)
```

```
plot(Longitude,
      Latitude,  pch=19,
      col=colors[rank(Magnitude)],
      main="Earthquake Locations",
      xlab="Longitude",
      ylab="Latitude") output:
```



Scenario 3: Wind Direction Analysis for Airport Operations: An airport collects hourly wind speed and wind direction for one year. Pilots need an intuitive visualization showing prevailing wind directions and seasonal variations. Develop a polar coordinate visualization (wind rose) to identify dominant wind directions and compare it with a Cartesian plot. Evaluate which visualization supports operational decision-making better.

Python code:

```
import numpy as np import
```

```
pandas as pd import
```

```
matplotlib.pyplot as plt
```

```
# Generate sample data np.random.seed(10)
```

```
n = 365 * 24 # hourly data for one year
```

```
wind_speed = np.random.uniform(5, 50, n) wind_direction =
```

```
np.random.uniform(0, 360, n)
```

```
data = pd.DataFrame({
    "WindSpeed": wind_speed,
    "WindDirection": wind_direction
})
```

```
print(data.head())
```

```
#####
```

```
# 1. Cartesian Scatter Plot
```

```
#####
```

```
plt.figure(figsize=(9,5)) plt.scatter(data.index,
    data["WindDirection"],
    s=5,
    color="blue")
```

```
plt.title("Wind Direction Throughout the Year") plt.xlabel("Hour") plt.ylabel("Wind Direction
(Degrees)") plt.grid(True) plt.show()
```

```
#####
```

```
# 2. Polar Plot (Wind Rose Style)
```

```
#####
```

```
theta = np.deg2rad(data["WindDirection"])
```

```
fig = plt.figure(figsize=(8,8)) ax =
fig.add_subplot(111, projection='polar')
```

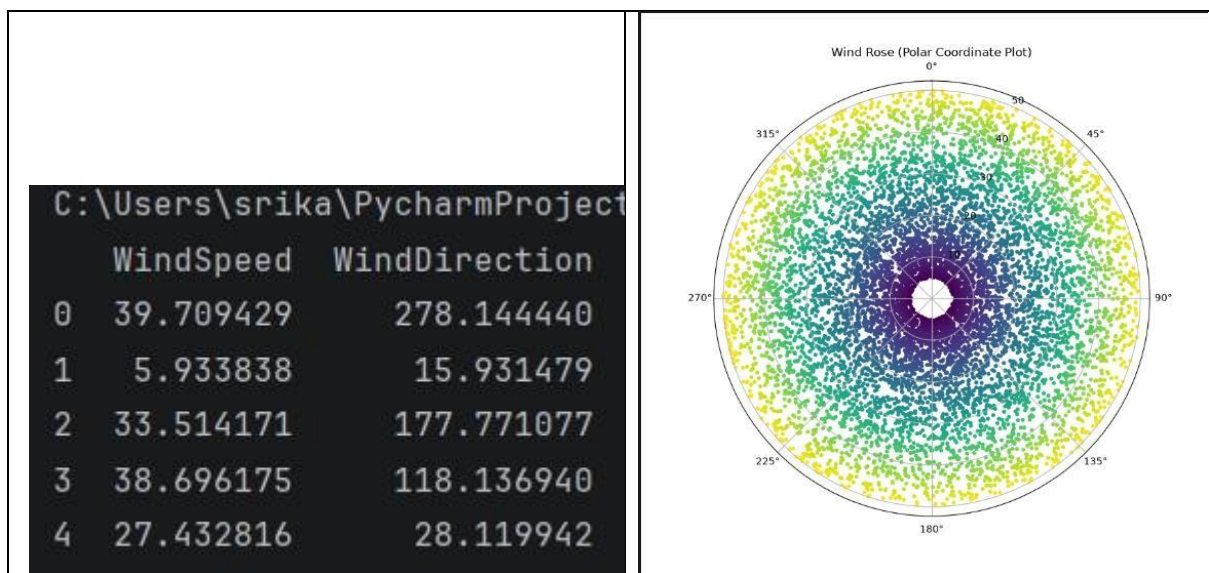
```

ax.scatter(theta,
data["WindSpeed"],
c=data["WindSpeed"],
cmap="viridis",      s=10)

ax.set_theta_zero_location("N") ax.set_theta_direction(-1)

plt.title("Wind Rose (Polar Coordinate Plot)")
plt.show() output:

```



R code:

```
# Wind Direction Analysis
```

```
set.seed(10)
```

```
n <- 365*24
```

```
WindSpeed <- runif(n,5,50)
```

```
WindDirection <- runif(n,0,360)
```

```
#####
```

```
# 1. Cartesian Plot
```

```
#####
```

```
plot(WindDirection,  
      type="p",  
      col="blue", pch=20,  
      main="Wind Direction  
Throughout the Year",  
      xlab="Observation",  
      ylab="Wind Direction  
(Degrees)")
```

```
#####
```

```
# 2. Polar Plot (Wind Rose Style)
```

```
#####
```

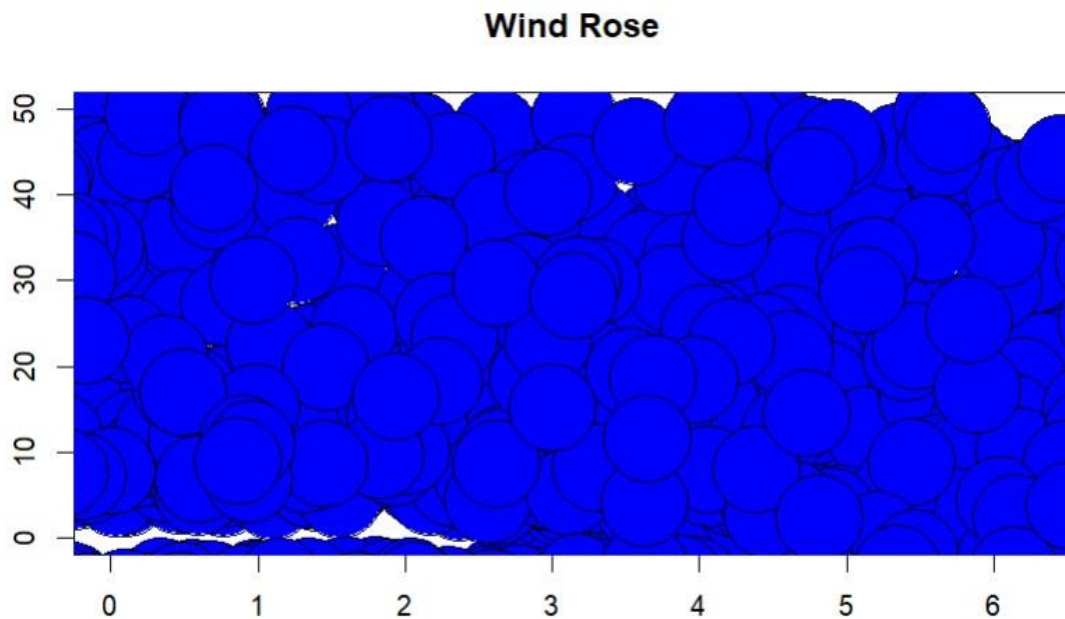
```
theta <- WindDirection*pi/180
```

```
plot(theta, WindSpeed,  
      type="n", xlim=c(0,2*pi),  
      ylim=c(0,max(WindSpeed)),  
      xlab="",  
      ylab="",  
      main="Wind Rose")
```

```

symbols(cos(theta)*WindSpeed,
sin(theta)*WindSpeed,
circles=rep(0.3,n),
inches=FALSE,    bg="blue",
add=TRUE) Output:

```



Scenario 4: Hospital Patient Monitoring Dashboard: ICU patients have heart rate, oxygen saturation, blood pressure, respiration rate, and temperature recorded every minute. Doctors require a dashboard that instantly highlights abnormal conditions. Build a dashboard using color as a highlighting tool. Select appropriate thresholds and explain why certain colors attract immediate attention without overwhelming users Python code:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

```

# Generate sample patient data
np.random.seed(10)

```

```

n = 100

```



```
data = pd.DataFrame({
    "HeartRate": np.random.randint(50, 140, n),
    "SpO2": np.random.randint(85, 101, n),
    "BloodPressure": np.random.randint(80, 180, n),
    "Respiration": np.random.randint(10, 35, n),
    "Temperature": np.round(np.random.uniform(35, 40, n),1)
})
```

```
#####
```

```
# Color Functions
```

```
#####
```

```
def hr_color(x):
    if x < 60 or x > 100:
        return 'red'
    return 'green'
```

```
def spo2_color(x):
    if x < 90:
        return 'red'
    elif x < 95:
        return 'orange'
    return 'green'
```

```
def bp_color(x):
    if x < 90 or x > 140:
```

```
        return 'red'
    return 'green'
```

```
def rr_color(x):
    if x < 12 or x > 20:
        return 'red'
    return 'green'
```

```
def temp_color(x):
    if x < 36 or x > 38:
        return 'red'
    return 'green'
```

```
#####
# Dashboard
#####
```

```
fig, axs = plt.subplots(3,2,figsize=(12,10))
```

```
# Heart Rate axs[0,0].bar(range(n),
    data["HeartRate"],      color=[hr_color(x)
for x in data["HeartRate"]]) axs[0,0].set_title("Heart
Rate")
```

```
# SpO2 axs[0,1].bar(range(n),      data["SpO2"],
    color=[spo2_color(x) for x in data["SpO2"]])
axs[0,1].set_title("Oxygen Saturation")
```

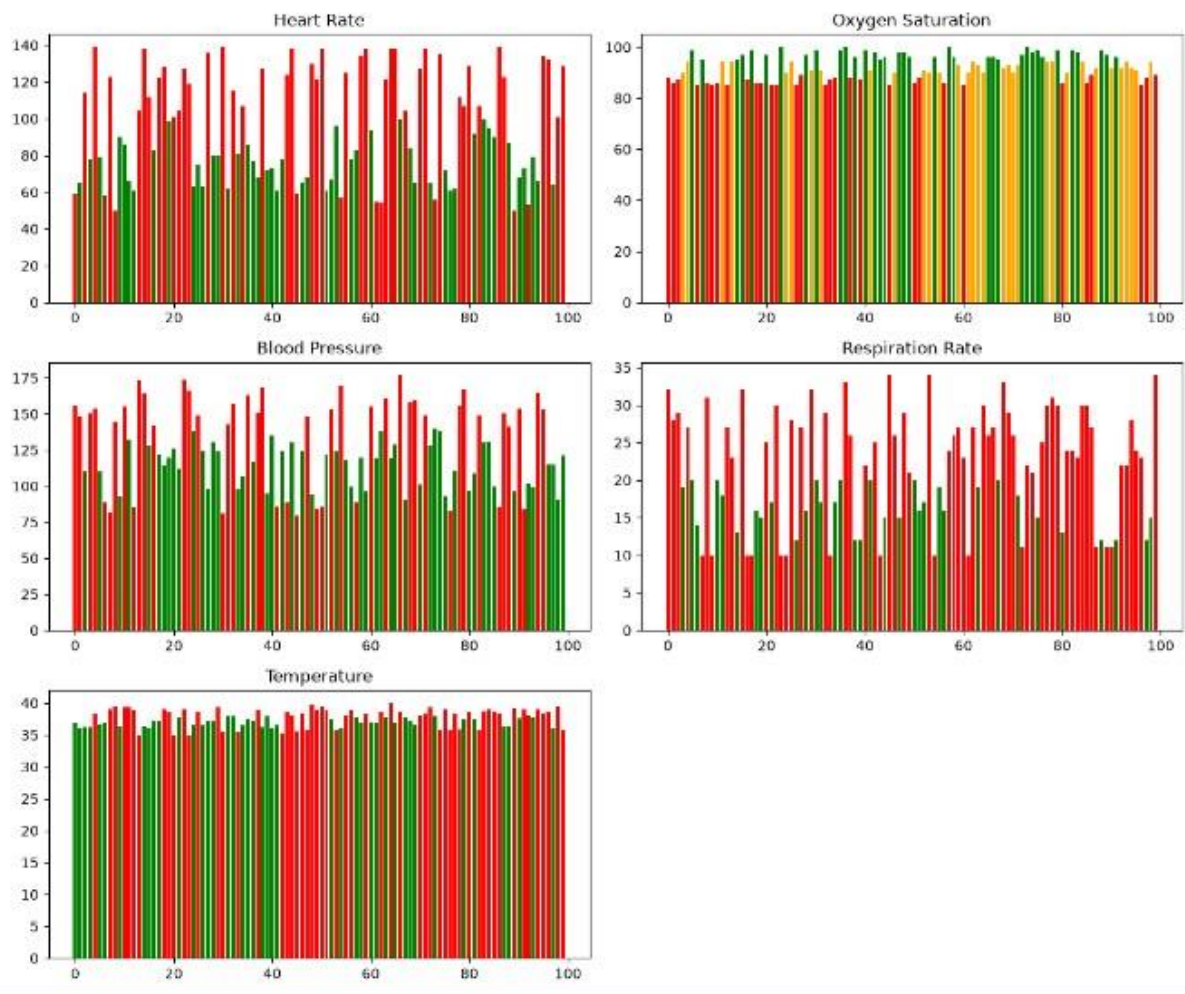
```
# Blood Pressure
axs[1,0].bar(range(n),      data["BloodPressure"],
color=[bp_color(x) for x in data["BloodPressure"]])
axs[1,0].set_title("Blood Pressure")

# Respiration axs[1,1].bar(range(n),
      data["Respiration"],      color=[rr_color(x)
for x in data["Respiration"]])
axs[1,1].set_title("Respiration Rate")

# Temperature axs[2,0].bar(range(n),
data["Temperature"],      color=[temp_color(x) for x in
data["Temperature"]]) axs[2,0].set_title("Temperature")

axs[2,1].axis("off")

plt.tight_layout()
plt.show() output:
```



R code:

```
set.seed(10)
```

```
n <- 100
```

```
HeartRate <- sample(50:140,n,replace=TRUE)
```

```
SpO2 <- sample(85:100,n,replace=TRUE)
```

```
BloodPressure <- sample(80:180,n,replace=TRUE)
```

```
Respiration <- sample(10:35,n,replace=TRUE)
```

```
Temperature <- round(runif(n,35,40),1)
```

```
par(mfrow=c(3,2))
```

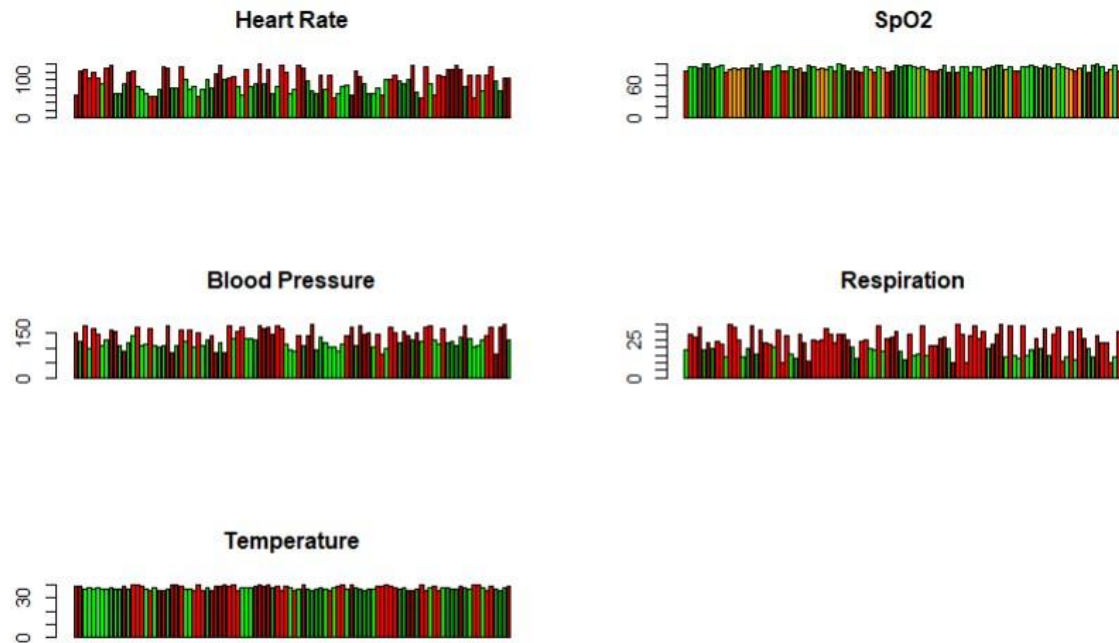
```
# Heart Rate colors <- ifelse(HeartRate<60 |  
HeartRate>100,"red","green")  
barplot(HeartRate,col=colors,main="Heart Rate")
```

```
# SpO2 colors <- ifelse(SpO2<90,"red",  
ifelse(SpO2<95,"orange","green"))  
barplot(SpO2,col=colors,main="SpO2")
```

```
# Blood Pressure colors <- ifelse(BloodPressure<90 |  
BloodPressure>140,  
"red","green")  
barplot(BloodPressure,col=colors,main="Blood Pressure")
```

```
# Respiration colors <- ifelse(Respiration<12 |  
Respiration>20,  
"red","green")  
barplot(Respiration,col=colors,main="Respiration")
```

```
# Temperature colors <- ifelse(Temperature<36 |  
Temperature>38,  
"red","green")  
barplot(Temperature,col=colors,main="Temperature") output:
```



Scenario 5:

Global Climate Change Study: Researchers analyze temperature anomalies across 180 countries from 1980–2025. The objective is to show both positive and negative deviations from historical averages. Design heatmaps using different diverging color palettes. Compare at least three palettes and justify which one communicates anomalies most effectively, including accessibility considerations.

Python code:

```
import numpy as np import
```

```
pandas as pd import
```

```
matplotlib.pyplot as plt
```

```
# Generate sample data np.random.seed(10)
```

```
countries = [f"Country_{i}" for i in range(1,181)] years =
list(range(1980,2026))
```

```
# Temperature anomalies (-3°C to +3°C) data =
np.random.uniform(-3,3,(len(countries),len(years)))
```

```
#####
```

```
# Heatmap 1 : Coolwarm
```

```
#####
```

```
plt.figure(figsize=(12,8)) plt.imshow(data,  
    aspect='auto',  
    cmap='coolwarm')
```

```
plt.colorbar(label="Temperature Anomaly (°C)")  
plt.title("Heatmap - Coolwarm Palette")  
plt.xlabel("Year") plt.ylabel("Countries")  
plt.xticks(range(0,len(years),5),  
    years[::5],  
    rotation=45) plt.show()
```

```
#####
```

```
# Heatmap 2 : RdBu
```

```
#####
```

```
plt.figure(figsize=(12,8)) plt.imshow(data,  
    aspect='auto',  
    cmap='RdBu')
```

```
plt.colorbar(label="Temperature Anomaly (°C)")  
plt.title("Heatmap - RdBu Palette")  
plt.xlabel("Year") plt.ylabel("Countries")  
plt.xticks(range(0,len(years),5),
```

```

        years[::5],
rotation=45) plt.show()

#####

# Heatmap 3 : PiYG

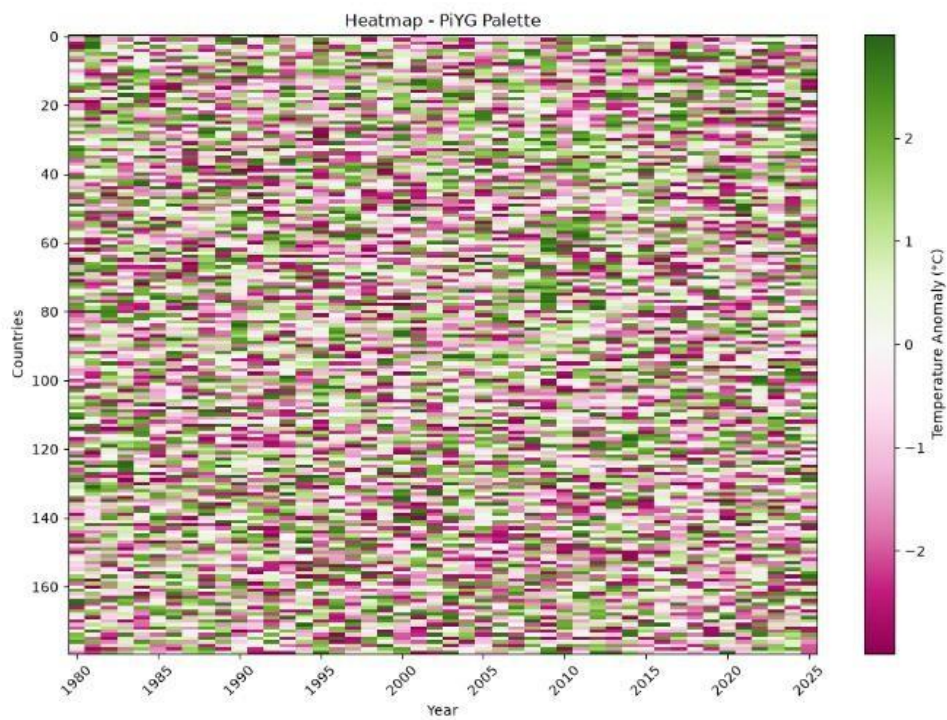
#####

plt.figure(figsize=(12,8)) plt.imshow(data,
        aspect='auto',
cmap='PiYG')

plt.colorbar(label="Temperature Anomaly (°C)")
plt.title("Heatmap - PiYG Palette")
plt.xlabel("Year") plt.ylabel("Countries")
plt.xticks(range(0,len(years),5),
        years[::5],
rotation=45) plt.show()

```

output:



R code:

```
set.seed(10)
```

```
countries <- 180 years
```

```
<- 1980:2025
```

```
data <- matrix(runif(countries*length(years),
```

```
  -3,
```

```
  3),
```

```
  nrow=countries)
```

```
#####
```

```
# Heatmap 1 : Blue-White-Red
```

```
#####
```

```
heatmap(data,
```

```
Colv=NA,    Rowv=NA,  
col=colorRampPalette(c("blue","white","red"))(100),  
scale="none",    main="Heatmap - Blue White Red")
```

```
#####
```

```
# Heatmap 2 : Purple-Green
```

```
#####
```

```
heatmap(data,  
    Colv=NA,    Rowv=NA,  
col=colorRampPalette(c("purple","white","green"))(100),  
scale="none",    main="Heatmap - Purple White Green")
```

```
#####
```

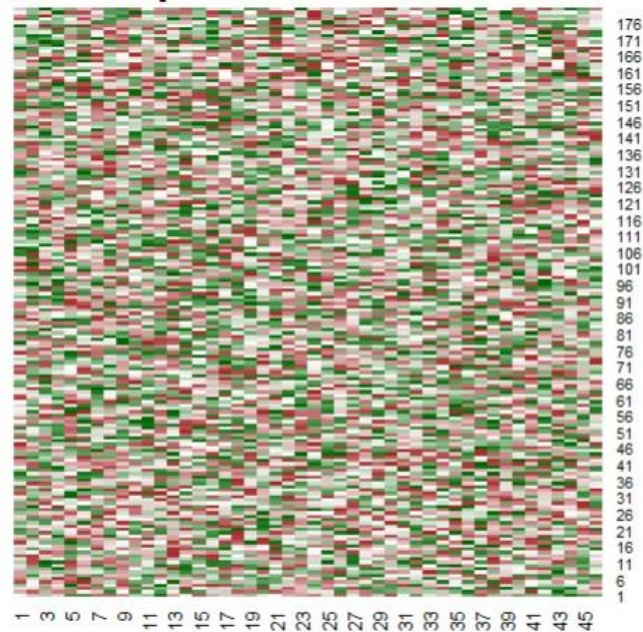
```
# Heatmap 3 : Brown-White-Green
```

```
#####
```

```
heatmap(data,  
    Colv=NA,    Rowv=NA,  
col=colorRampPalette(c("brown","white","darkgreen"))(100),  
scale="none",
```

```
    main="Heatmap - Brown White Green") output:
```

Heatmap - Brown White Green



Scenario 6:

E-Commerce Sales Analytics: A retail company sells over 50 product categories across 30 regions. Management wants to distinguish categories while simultaneously representing sales volume. Create visualizations that use color both to distinguish categories and represent numerical values. Discuss the challenges of combining qualitative and quantitative color encoding.

Python code:

```
import numpy as np import
```

```
pandas as pd import
```

```
matplotlib.pyplot as plt
```

```
# Generate sample data np.random.seed(10)
```

```
categories = [f"Category_{i}" for i in range(1, 51)]
```

```
regions = [f"Region_{i}" for i in range(1, 31)]
```

```
data = pd.DataFrame({
```

```

"Category": np.random.choice(categories, 500),
"Region": np.random.choice(regions, 500),
"Sales": np.random.randint(1000, 100000, 500)
})

print(data.head())

#####

# 1. Scatter Plot

#####

unique_categories = data["Category"].unique() color_map =
plt.cm.tab20(np.linspace(0, 1, len(unique_categories)))

category_colors = { cat: color_map[i %
len(color_map)] for i, cat in
enumerate(unique_categories)
}

plt.figure(figsize=(12,6))

for cat in unique_categories:
    subset = data[data["Category"] == cat]
    plt.scatter(    subset["Region"],

        subset["Sales"],
        color=category_colors[cat],
        alpha=0.7,    label=cat

```

```
)
```

```
plt.xticks(rotation=90) plt.title("Sales by Region  
and Product Category") plt.xlabel("Region")  
plt.ylabel("Sales") plt.tight_layout() plt.show()
```

```
#####
```

```
# 2. Heatmap (Sales Volume)
```

```
#####
```

```
pivot = data.pivot_table(  
    values="Sales",  
    index="Category",  
    columns="Region",  
    aggfunc="sum", fill_value=0  
)
```

```
plt.figure(figsize=(12,10))
```

```
plt.imshow(pivot,  
            cmap="YlOrRd",  
            aspect="auto")
```

```
plt.colorbar(label="Sales Volume")
```

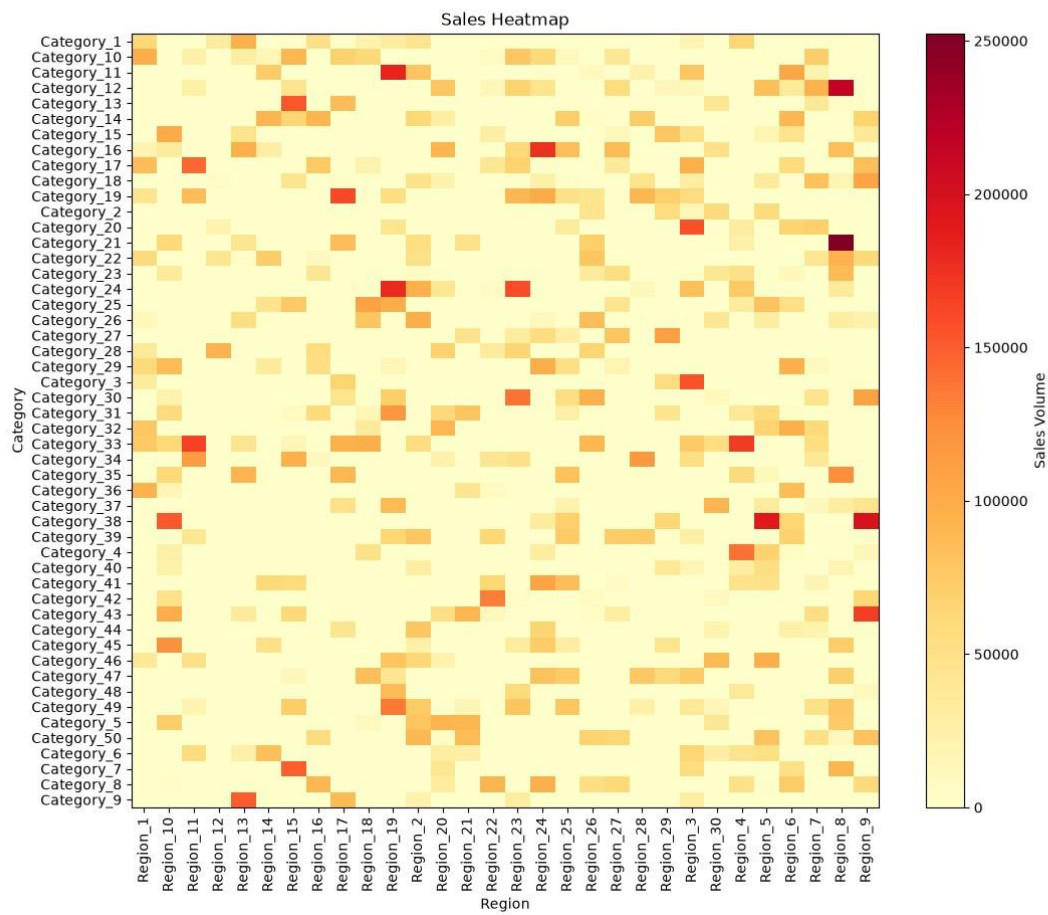
```
plt.xticks(range(len(pivot.columns)),  
            pivot.columns, rotation=90)
```

```
plt.yticks(range(len(pivot.index)),
           pivot.index)
```

```
plt.title("Sales Heatmap")
```

```
plt.xlabel("Region") plt.ylabel("Category")
```

```
plt.show() Output:
```



R code:

```
set.seed(10)
```

```
categories <- paste("Category",1:50) regions
```

```
<- paste("Region",1:30)
```

```
n <- 500
```

```
Category <- sample(categories,n,replace=TRUE)
```

```
Region <- sample(regions,n,replace=TRUE)
```

```
Sales <- sample(1000:100000,n,replace=TRUE)
```

```
#####
```

```
# Scatter Plot
```

```
#####
```

```
category_colors <- rainbow(length(categories))
```

```
plot(as.numeric(factor(Region)),
```

```
     Sales,    pch=19,
```

```
     col=category_colors[as.numeric(factor(Category))],
```

```
     xaxt="n",  xlab="Region",  ylab="Sales",
```

```
     main="Sales by Region")
```

```
axis(1,
```

```
     at=1:length(unique(Region)),
```

```
     labels=unique(Region),  las=2,
```

```
     cex.axis=0.6)
```

```
#####
```

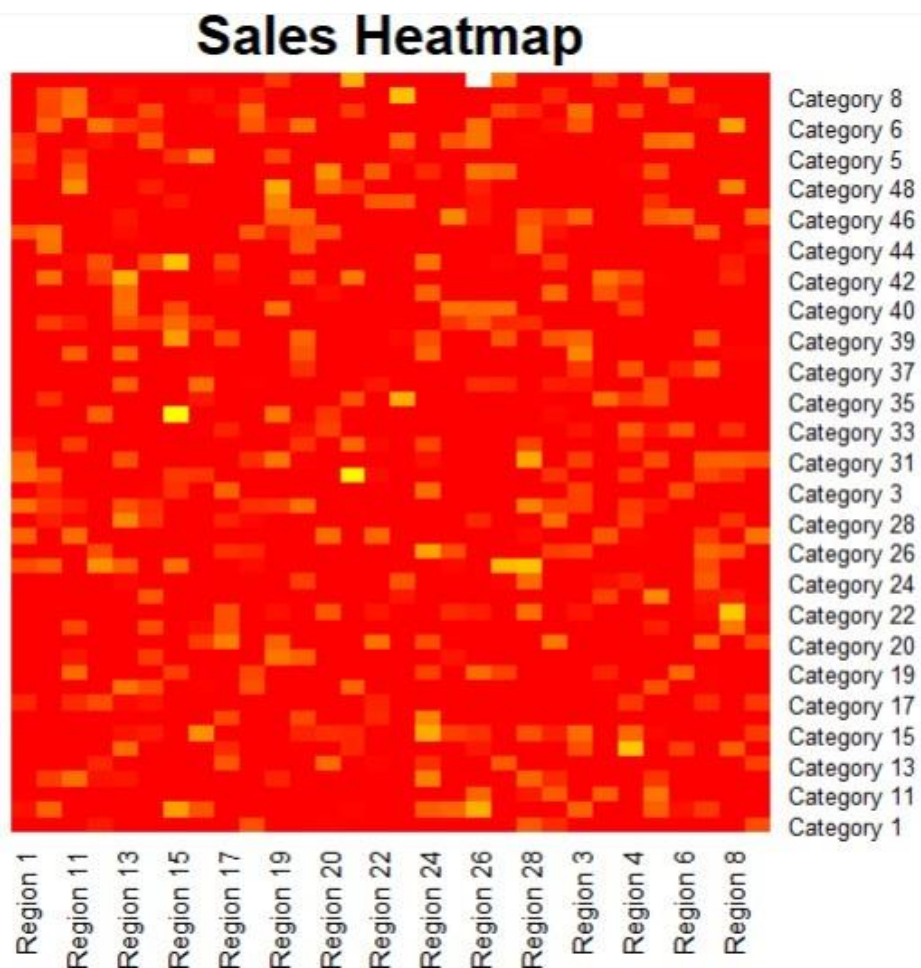
```
# Heatmap
```

```
#####
```

```
sales_matrix <- xtabs(Sales ~ Category + Region)
```

```
heatmap(
  sales_matrix,
  Rowv=NA, Colv=NA,
  scale="none",
  col=heat.colors(100),
  main="Sales Heatmap"
)
```

Output:



Scenario 7:

Satellite Image Vegetation Analysis: A satellite dataset provides NDVI values ranging from -1 to $+1$. Environmental scientists must identify healthy vegetation, barren land, and water bodies. Apply different sequential and diverging color palettes to represent vegetation health. Evaluate which palette minimizes interpretation errors and improves readability.

Python code:

```
import numpy as np import
```

```
matplotlib.pyplot as plt
```

```
# Generate sample NDVI data np.random.seed(10)
```

```
ndvi = np.random.uniform(-1, 1, (100, 100))
```

```
#####
```

```
# 1. Sequential Palette (Greens)
```

```
#####
```

```
plt.figure(figsize=(6,5))
```

```
plt.imshow(ndvi,
```

```
cmap="Greens",
```

```
vmin=-1,      vmax=1)
```

```
plt.colorbar(label="NDVI") plt.title("NDVI -
```

```
Sequential Palette (Greens)") plt.show()
```

```
#####
```

```
# 2. Diverging Palette (RdYlGn)
```

```
#####
```

```
plt.figure(figsize=(6,5))
```

```
plt.imshow(ndvi,
```

```
cmap="RdYlGn",
```

```
vmin=-1,      vmax=1)
```

```
plt.colorbar(label="NDVI") plt.title("NDVI - Diverging  
Palette (Red-Yellow-Green)") plt.show()
```

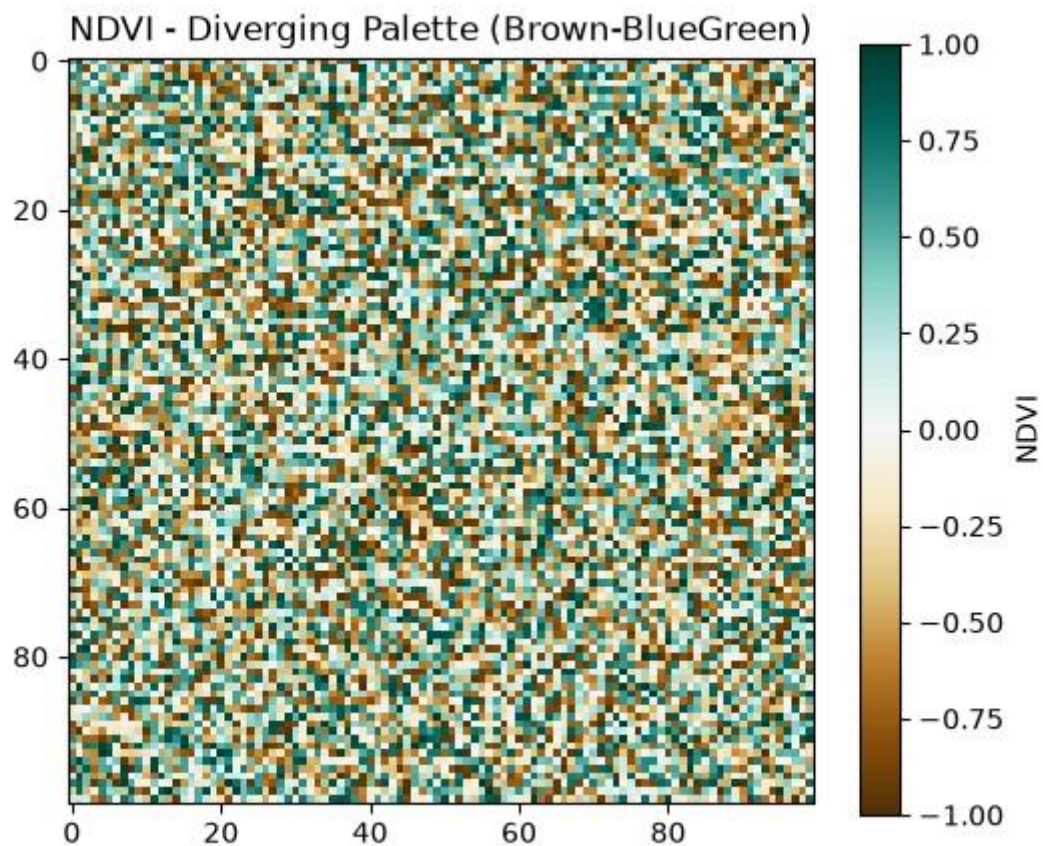
```
#####
```

```
# 3. Diverging Palette (BrBG)
```

```
#####
```

```
plt.figure(figsize=(6,5)) plt.imshow(ndvi,  
    cmap="BrBG",  
vmin=-1,    vmax=1)
```

```
plt.colorbar(label="NDVI") plt.title("NDVI - Diverging  
Palette (Brown-BlueGreen)") plt.show() output:
```



R code: set.seed(10)

```
ndvi <- matrix(runif(100*100,-1,1),100,100)
```

```
#####
```

```
# 1. Sequential Palette (Greens)
```

```
#####
```

```
image(
```

```
  ndvi, col=colorRampPalette(c("white","lightgreen","darkgreen"))(100),
```

```
  main="NDVI - Sequential Greens"
```

```
)
```

```
#####
```

```
# 2. Diverging Palette (Blue-White-Green)
```

```
#####
```

```
image(  
  ndvi, col=colorRampPalette(c("blue","white","green"))(100),  
  main="NDVI - Diverging Blue-White-Green"  
)
```

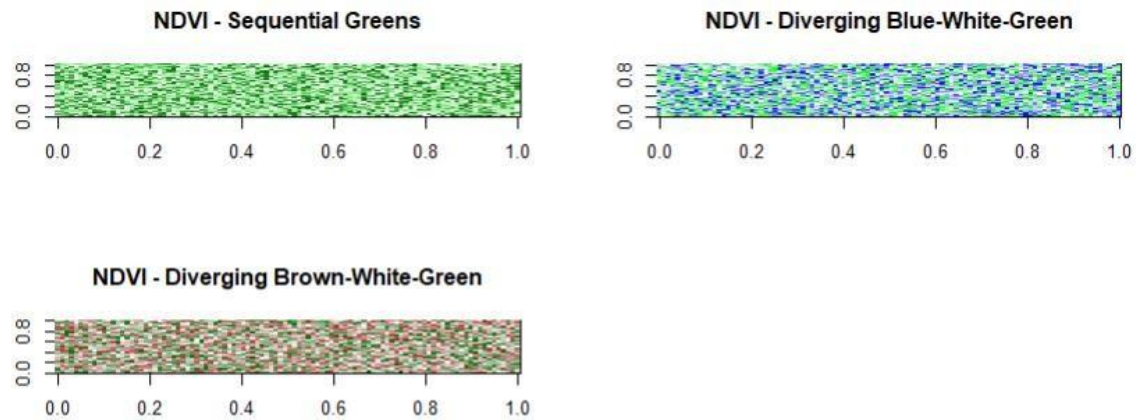
```
#####
```

```
# 3. Diverging Palette (Brown-White-Green)
```

```
#####
```

```
image(  
  ndvi,  
  col=colorRampPalette(c("brown","white","darkgreen"))(100), main="NDVI  
- Diverging Brown-White-Green"  
)
```

Output:



Scenario 8:

Cryptocurrency Market Volatility: Daily prices of Bitcoin, Ethereum, Solana, and Cardano span several years. Price values differ by several orders of magnitude, making direct comparison difficult. Compare Cartesian plots with logarithmic axes. Use color strategically to distinguish cryptocurrencies while maintaining visual clarity. Explain the benefits and limitations of nonlinear axes.

Python code:

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Generate sample data np.random.seed(10)
```

```
days = pd.date_range("2020-01-01", periods=500)
```

```
btc = np.random.uniform(20000,120000,500)
```

```
eth = np.random.uniform(500,8000,500) sol =
```

```
np.random.uniform(10,300,500)
```

```
ada = np.random.uniform(0.2,3,500)
```

```
#####
```

```
# Linear (Cartesian) Plot
```

```
#####
```

```
plt.figure(figsize=(12,6))
```

```
plt.plot(days, btc, label="Bitcoin", color="orange") plt.plot(days,  
eth, label="Ethereum", color="blue") plt.plot(days, sol,  
label="Solana", color="green") plt.plot(days, ada,  
label="Cardano", color="red")
```

```
plt.title("Cryptocurrency Prices (Linear Scale)")
```

```
plt.xlabel("Date") plt.ylabel("Price (USD)")
```

```
plt.legend() plt.grid(True) plt.show()
```

```
#####
```

```
# Logarithmic Scale Plot
```

```
#####
```

```
plt.figure(figsize=(12,6))
```

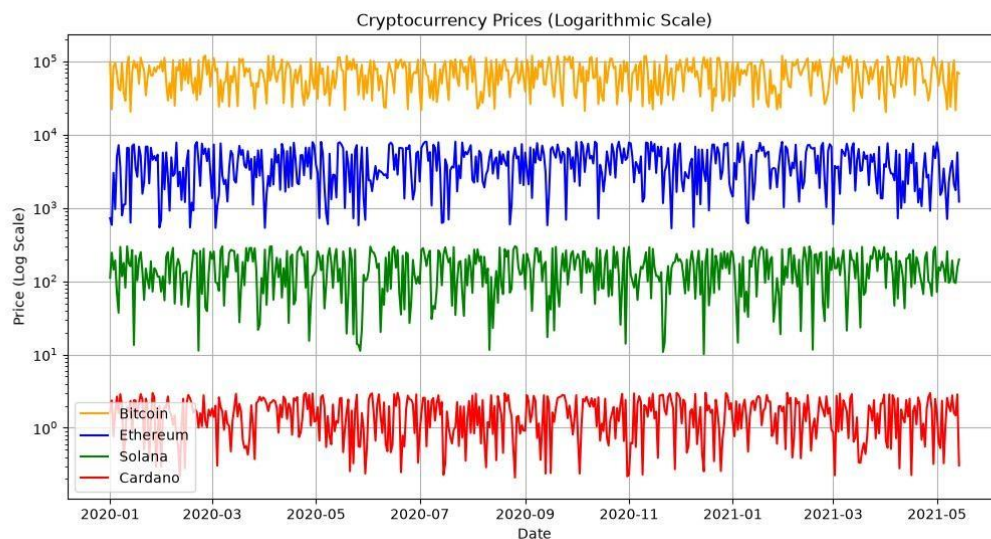
```
plt.plot(days, btc, label="Bitcoin", color="orange") plt.plot(days,  
eth, label="Ethereum", color="blue") plt.plot(days, sol,  
label="Solana", color="green") plt.plot(days, ada, label="Cardano",  
color="red")
```

```
plt.yscale("log")
```

```
plt.title("Cryptocurrency Prices (Logarithmic Scale)")
```

```
plt.xlabel("Date") plt.ylabel("Price (Log Scale)")
```

```
plt.legend() plt.grid(True) plt.show() Output:
```



R code:

```
set.seed(10)
```

```
days <- 1:500
```

```
Bitcoin <- runif(500,20000,120000)
```

```
Ethereum <- runif(500,500,8000)
```

```
Solana <- runif(500,10,300)
```

```
Cardano <- runif(500,0.2,3)
```

```
#####
```

```
# Linear Plot
```

```
#####
```

```

plot(days,
      Bitcoin, type="l", col="orange",
      ylim=c(0,max(Bitcoin)), xlab="Day",
      ylab="Price (USD)", main="Cryptocurrency
Prices (Linear Scale)")

```

```

lines(days,Ethereum,col="blue") lines(days,Solana,col="green")
lines(days,Cardano,col="red")

```

```

legend("topright",
legend=c("Bitcoin","Ethereum","Solana","Cardano"),
col=c("orange","blue","green","red"),
lty=1)

```

```
#####
```

```
# Logarithmic Plot #####
```

```

plot(days,
      Bitcoin, type="l", log="y",
      col="orange", xlab="Day", ylab="Log
Price", main="Cryptocurrency Prices (Log
Scale)")

```

```

lines(days,Ethereum,col="blue") lines(days,Solana,col="green")
lines(days,Cardano,col="red")

```

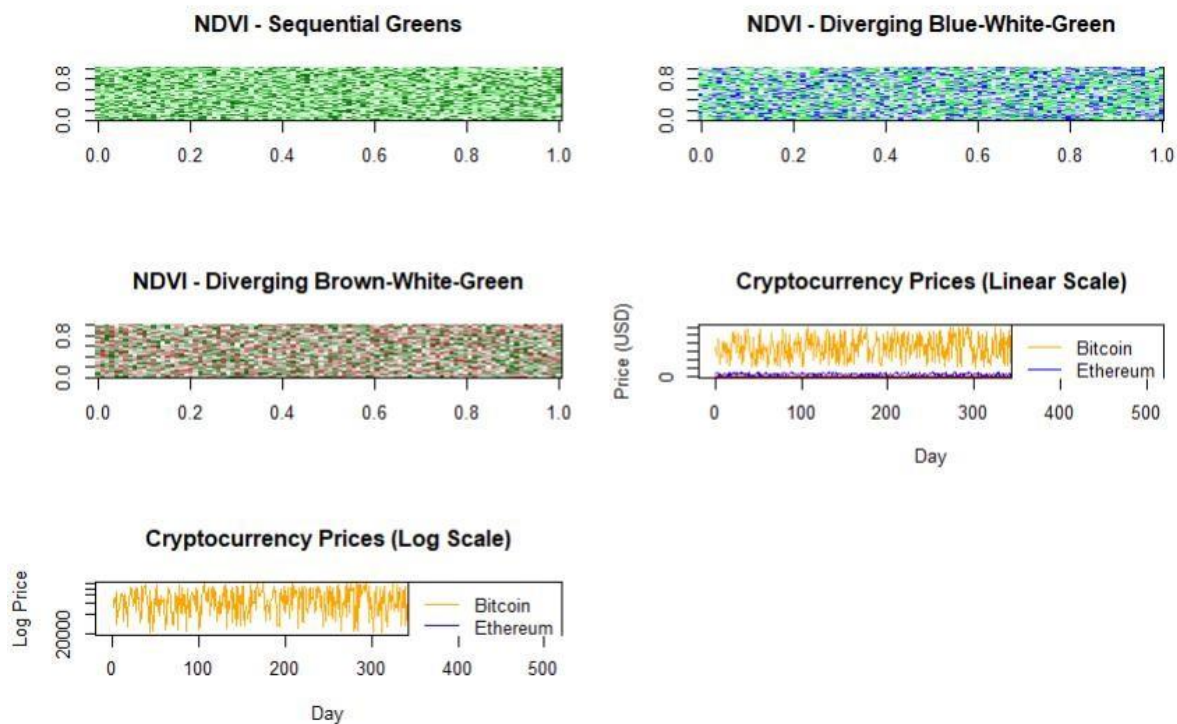


```

legend("topright",
legend=c("Bitcoin","Ethereum","Solana","Cardano"),
col=c("orange","blue","green","red"),
lty=1)

```

Output:



Scenario 9:

Marathon Runner Performance Analysis: A marathon organizer records runner speed, heart rate, distance covered, elevation, and running direction throughout the race. Coaches want to analyze pacing strategies and directional movement. Develop both Cartesian and polar coordinate visualizations. Use color gradients to represent speed variations and recommend which visualization provides deeper performance insights.

Python code:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Generate sample marathon data
np.random.seed(10)

```

```
n = 300
```

```
distance = np.linspace(0, 42.2, n)
```

```
speed = np.random.uniform(8, 18, n) heart_rate =
```

```
np.random.randint(110, 190, n) elevation =
```

```
np.random.uniform(50, 250, n) direction =
```

```
np.random.uniform(0, 360, n)
```

```
#####
```

```
# Cartesian Plot
```

```
#####
```

```
plt.figure(figsize=(10,6))
```

```
scatter = plt.scatter(distance,
```

```
                        elevation,
```

```
c=speed,
```

```
cmap="viridis",
```

```
s=60)
```

```
plt.colorbar(scatter,label="Speed (km/h)")
```

```
plt.title("Distance vs Elevation")
```

```
plt.xlabel("Distance (km)")
```

```
plt.ylabel("Elevation (m)") plt.grid(True)
```

```
plt.show()
```

```
#####
```

```

# Polar Coordinate Plot

##### theta = np.deg2rad(direction)

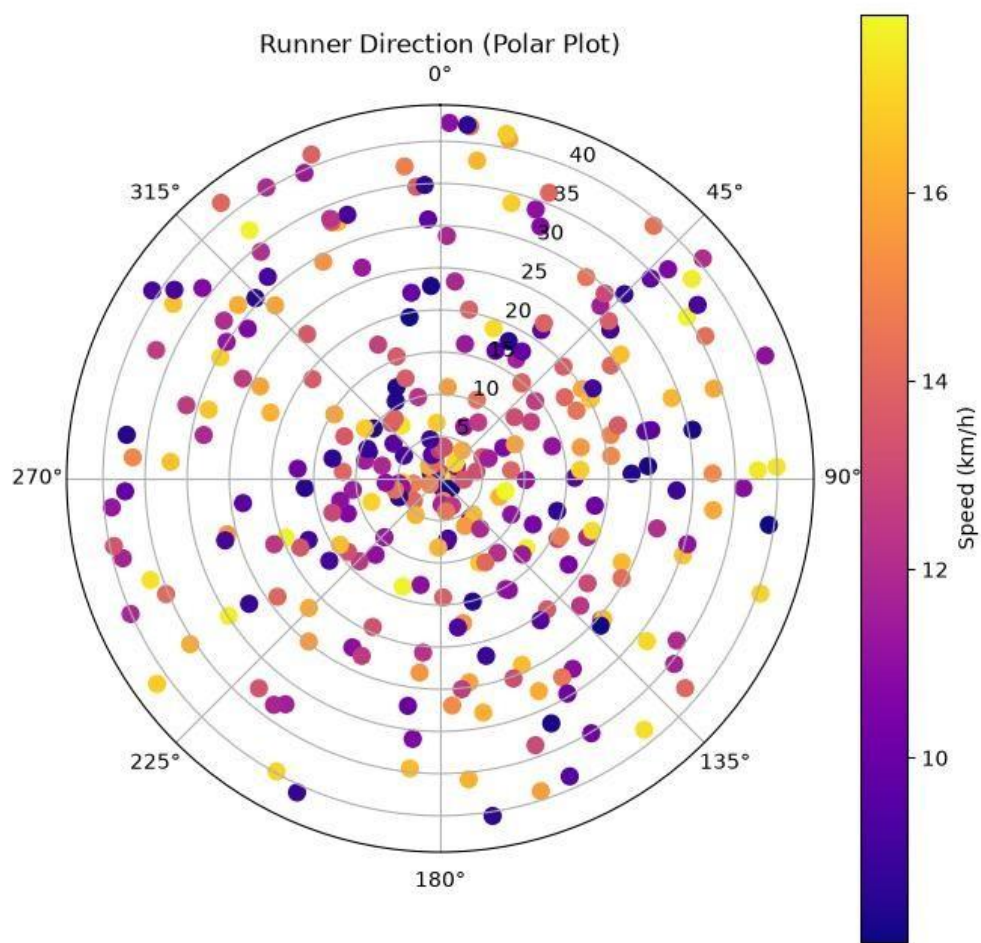
fig = plt.figure(figsize=(8,8)) ax =
fig.add_subplot(111, projection='polar')

scatter = ax.scatter(theta,
                      distance,
                      c=speed,
                      cmap="plasma",
                      s=60)

ax.set_theta_zero_location("N") ax.set_theta_direction(-1)

plt.colorbar(scatter,label="Speed (km/h)")
plt.title("Runner Direction (Polar Plot)")
plt.show() Output:

```



R code; set.seed(10)

n <- 300

Distance <- seq(0,42.2,length.out=n)

Speed <- runif(n,8,18)

HeartRate <- sample(110:190,n,replace=TRUE)

Elevation <- runif(n,50,250)

Direction <- runif(n,0,360)

```
#####
```

```
# Cartesian Plot
```

```
#####
```

```
colors <- heat.colors(n)
```

```
plot(Distance,  
      Elevation, pch=19,  
      col=colors[rank(Speed)],  
      main="Distance vs Elevation",  
      xlab="Distance (km)",  
      ylab="Elevation (m)")
```

```
#####
```

```
# Polar Plot
```

```
#####
```

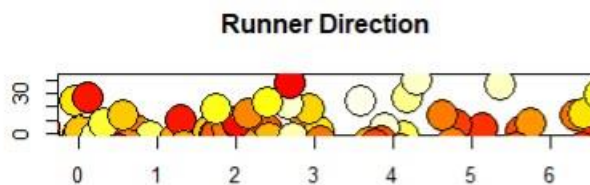
```
theta <- Direction*pi/180
```

```
plot(theta, Distance,  
      type="n", xlim=c(0,2*pi),  
      ylim=c(0,max(Distance)),  
      xlab="",  
      ylab="",  
      main="Runner Direction")
```

```

symbols(cos(theta)*Distance,
sin(theta)*Distance,
circles=rep(0.2,n),
inches=FALSE,
bg=colors[rank(Speed)],
add=TRUE) Output:

```



Scenario 10:

COVID-19 Vaccination Campaign Dashboard: Health officials monitor vaccination rates across districts, age groups, and vaccine types. They need visualizations suitable for both policymakers and the general public. Design two visualization versions using different color palettes and highlighting strategies. Evaluate accessibility, interpretability, and effectiveness for audiences with color vision deficiencies.

Python code:

```
import pandas as pd import
```

```
numpy as np import
```

```
matplotlib.pyplot as plt
```

```
# Generate sample data np.random.seed(10)
```

```
districts = [f"District_{i}" for i in range(1,21)]
```

```
age_groups = ["0-18","19-40","41-60","60+"]
```

```
vaccines = ["Covishield","Covaxin","Sputnik"]
```

```
n = 200
```

```
data = pd.DataFrame({
    "District": np.random.choice(districts,n),
    "AgeGroup": np.random.choice(age_groups,n),
    "Vaccine": np.random.choice(vaccines,n),
    "VaccinationRate": np.random.randint(40,100,n)
})
```

```
#####
```

```
# Version 1 : Policymakers Dashboard
```

```
#####
```

```
pivot = data.pivot_table(values="VaccinationRate",
                           index="District",
                           columns="AgeGroup",
                           aggfunc="mean")
```

```
plt.figure(figsize=(10,8))
```

```
plt.imshow(pivot,
            cmap="RdYlGn",
            aspect="auto")
plt.colorbar(label="Vaccination Rate
(%)")
```

```
plt.xticks(range(len(pivot.columns)),pivot.columns)
plt.yticks(range(len(pivot.index)),pivot.index)
```

```
plt.title("Vaccination Dashboard (Policy Makers)") plt.show()
```

```
#####
```

```
# Version 2 : Public Dashboard
```

```
#####
```

```
avg = data.groupby("Vaccine")["VaccinationRate"].mean()
```

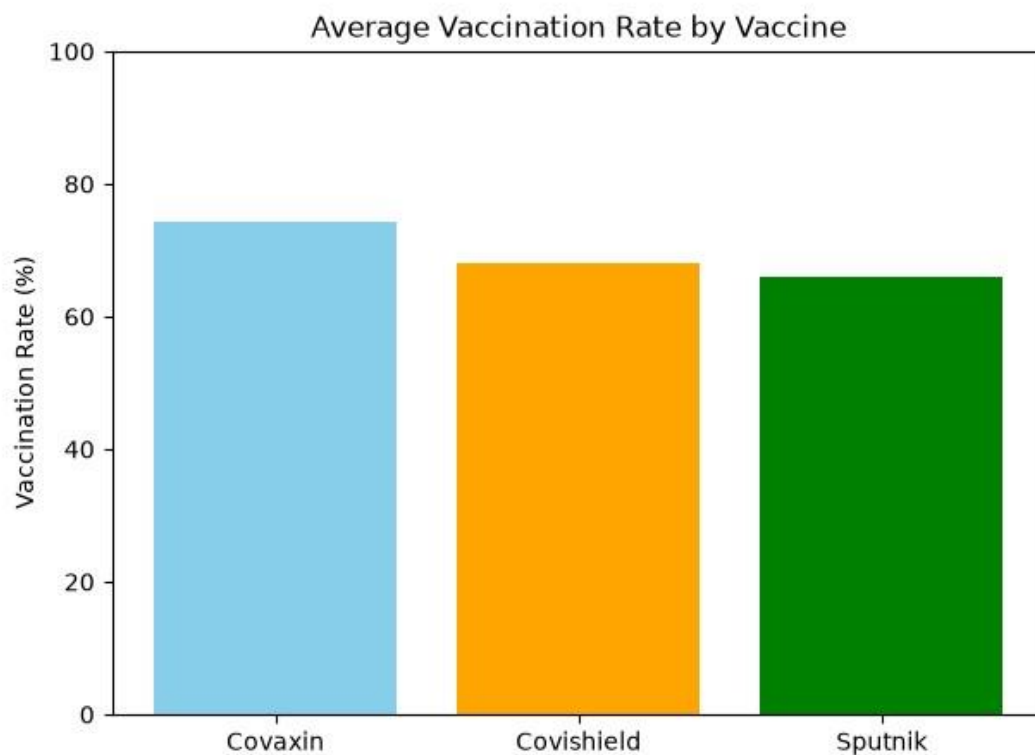
```
plt.figure(figsize=(7,5))
```

```
plt.bar(avg.index,      avg.values,  
color=["skyblue","orange","green"])
```

```
plt.ylim(0,100)
```

```
plt.title("Average Vaccination Rate by Vaccine") plt.ylabel("Vaccination  
Rate (%)")
```

```
plt.show() output;
```

R code:

```
set.seed(10)
```

```
districts <- paste("District",1:20) agegroups <-  
c("0-18","19-40","41-60","60+") vaccines <-  
c("Covishield","Covaxin","Sputnik")
```

```
n <- 200
```

```
District <- sample(districts,n,replace=TRUE)  
AgeGroup <- sample(agegroups,n,replace=TRUE)  
Vaccine <- sample(vaccines,n,replace=TRUE)  
VaccinationRate <- sample(40:100,n,replace=TRUE)
```

```
#####
```

```
# Dashboard 1 : Heatmap
```

```
#####
```

```
mat <- xtabs(VaccinationRate~District+AgeGroup)
```

```
heatmap(mat,
```

```
      Rowv=NA,      Colv=NA,      scale="none",
```

```
col=colorRampPalette(c("red","yellow","green"))(100),
```

```
main="Policy Dashboard")
```

```
#####
```

```
# Dashboard 2 : Public Dashboard
```

```
#####
```

```
avg <- tapply(VaccinationRate,
```

```
      Vaccine,
```

```
mean)
```

```
barplot(avg,
```

```
col=c("skyblue","orange","green"),
```

```
ylim=c(0,100),      main="Vaccination
```

```
Rate") output:
```

